A Project Report
on

A Secure Multi-Disease Medical Image ClassificationUsing Convolutional Neural Networks and Fully Homomorphic Encryption

Department of CSE

submitted in partial fulfillment for the award of

Bachelor of Technology
in
Computer Science & Engineering
by

V. Harshitha (Y22ACS589) **P. Hema Santhosh Koti Reddy (Y22ACS539)**

T. BhanuPrakash (Y22ACS577) **R. Poojya Vyshnavi (Y22ACS542)**

K. Sahithi (L21ACS411)

Under the guidance of
M. Karuna, M Tech
Assistant Professor

Department of Computer Science and Engineering
Bapatla Engineering College
(Autonomous)
(Affiliated to Acharya Nagarjuna University)
BAPATLA – 522102, Andhra Pradesh, INDIA

2025-2026

Department of
Computer Science & Engineering

CERTIFICATE

This is to certify that the project report entitled **A Secure Multi-Disease Medical Image Classification Using Convolutional Neural Networks and Fully Homomorphic Encryption** that is being submitted by V Harshitha (Y22ACS589), P Hema Santhsoh Koti Reddy (Y22ACS539), T Bhanu Prakash (Y22ACS577), R Poojya Vyshnavi (Y22ACS542), K Sahithi (L21ACS411) in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science & Engineering to the Acharya Nagarjuna University is a record of bonafide work carried out by them under our guidance and supervision.

Date:

Signature of the Guide	**Signature of the HOD**
M. Karuna	**Dr. M. Rajesh Babu**
Assistant Professor	**Assoc. Prof. & Head**

DECLARATION

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

V Harshitha (Y22ACS589)
P Hema Santhosh Koti Reddy (Y22ACS539)
T Bhanu prakash (Y22ACS577)
R Poojya Vyshnavi (Y22ACS542)
K Sahiti (L21ACS411)

ACKNOWLEDGEMENT

We sincerely thank the following distinguished personalities who have offered their valuable advice and support for the successful completion of this work.

We are deeply indebted to our most respected guide, **Medikonda Karuna, Assistant Professor**, Department of Computer Science and Engineering, for her valuable guidance, constant encouragement, constructive suggestions, and inspiring support throughout the course of this project.

We extend our sincere thanks to **Dr. M. Rajesh Babu, Associate Professor & Head of the Department**, Department of Computer Science and Engineering, for his cooperation and for providing the necessary facilities and resources to carry out this work successfully.

We would also like to express our heartfelt gratitude to our beloved **Principal, Dr. B. Srinivasarao**, for providing the required infrastructure, online resources, and institutional support to complete this project.

We express our sincere thanks to our **Project Coordinators, Dr. C. Prakasa Rao, Professor, Department of CSE, and Dr. R. Venkata Lakshmi, Assistant Professor, Department of CSE**, for their valuable suggestions and guidance in preparing and presenting this document.

We are always thankful to our **parents and family members** for their continuous support, encouragement, and motivation. Without their support, we would not have been able to complete this project work successfully.

Finally, we extend our sincere gratitude to all the teaching faculty and non-teaching staff of the Department of Computer Science and Engineering who helped us directly or indirectly with their cooperation and encouragement.

V Harshitha (Y22ACS589)

P Hema Santhosh Koti Reddy (Y22ACS539)

T Bhanu Prakash (Y22ACS577)

R Poojya Vyshnavi (Y22ACS542)

K Sahiti (L21ACS411)

Table of Contents

List of Figures ix

List of Tables x

ABSTRACT xii

1. INTRODUCTION 1

1.1 Motivation 1

1.2 Problem Definition 2

1.3 Objective of Project 2

1.4 Scope 3

1.5 Deep Learning and Convolutional Neural Networks 3

1.5.1 Deep Learning Paradigms 3

1.5.2 Neural Networks 5

1.5.3 Convolutional Neural Networks (CNNs) 6

1.5.4 Training Strategy 7

1.6 Fully Homomorphic Encryption 7

1.6.1 Motivation for Encrypted Computation 7

1.6.2 Homomorphic Encryption — Core Concept 7

1.6.3 Evolution of FHE Schemes 7

1.6.4 CKKS Encryption Lifecycle 8

1.6.5 FHE-Compatible Neural Networks 9

1.6.6 TenSEAL Library 10

2. LITERATURE SURVEY 11

2.1 Existing System 11

2.1.1 Overview 11

2.1.2 Architecture and Methodology 11

2.1.3 Key Technical Innovations 12

2.1.4 Experimental Results and Performance 13

2.2 Disadvantages of Existing System 13

2.3 Proposed System 14

2.3.1Overall System Architecture 14

2.3.2 Disease Classes and Medical Information 14

2.3.3 SimpleCNN Architecture 15

2.3.4 FHEPolynomialHead Architecture 16

2.3.5 CKKS Parameters 16

2.4 Advantages of Proposed System 17

3.ANALYSIS 18

3.1 Software Requirement Specification 18

3.1.1 User Requirement 18

3.1.2 Software Requirements 18

3.2 System Architecture 19

4. DESIGN 22

4.1 Introduction 22

4.2 DFD / UML Diagrams 22

4.2.1 Data Flow Diagram – Level 0 (Context Diagram) 22

4.2.2 Data Flow Diagram – Level 1 22

4.2.3 UML Sequence Diagram – FHE Inference Pipeline 23

4.3 Data Dictionary 26

4.4 Key Libraries and Frameworks 27

4.4.1 PyTorch 27

4.4.2 TenSEAL 27

4.4.3 Flask 27

4.4.4 NumPy 28

4.4.5 OpenCV (CV2) 28

4.4.6 Scikit-Learn 28

4.4.7 Matplotlib and Seaborn 28

5. IMPLEMENTATION 29

5.1 Method of Implementation 29

5.1.1 Implementation of Algorithms 29

5.2 Module Implementation Details 30

5.2.1 train.py — Training Pipeline 30

5.2.2 fhe_model.py — FHE Classifier 31

5.2.3 app.py — Flask Web Application 31

5.3 Encrypted Inference End-to-End Flow 32

6. RESULTS 33

6.1 CNN Model Performance 33

6.2 FHE Inference Performance 33

6.3 Evaluation of performance metrics 36

6.4 Web Application Interface 36

7. TESTING & VALIDATION 39

7.1 Features to be tested 39

7.2 Features not to be tested 39

7.3 Testing Tools And Environment 39

8. CONCLUSION 40

9. FUTURE ENHANCEMENT 41

10. BIBLIOGRAPY 42

List of Figures

Figure 1.1 Convolutional Neural Network diagram for Image Classification…..…..3

Figure 1.2 Recurrent Neural Network diagram……...4

Figure 1.3 Deep Neural Network diagram……..4

Figure 1.4 Architecture of Neural Networks……...5

Figure 1.5 CNN Architecture for Chest X-ray Image Classification ……..6

Figure 3.1 System Architecture …….20

Figure 4.1 Level 0 DFD…….…….22

Figure 4.2 Level 1 DFD…….…….23

Figure 4.3 UML Sequence Diagram…….……..24

Figure 4.4 Use Case Diagram…….25

Figure 6.1 Per-Class Accuracy Comparison……...35

Figure 6.2 Performance Metrics Comparison…….35

Figure 6.3 Confusion Matrix……...36

Figure 6.4 Home Page…….37

Figure 6.5 Prediction and Shows Possible Causes……..37

Figure 6.6 Recommended Precautions…..……..38

Figure 6.7 Download the Result……..38

List of Tables

Table 1.1 CKKS Encryption Parameters Used in this project ……8

Table 2.1 Numbers of Images in Dataset ……15

Table 2.2 SimpleCNN Layer-by-Layer Architecture……..16

Table 3.1 Software Requirements……...18

Table 3.2 Hardware Requirements……..19

Table 4.1 Data Dictionary…….....26

Table 6.1 Per-Class Accuracy Comparison ……34

Table 6.2 Performance Metrics Comparison……...35

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
CKKS	Cheon-Kim-Kim-Song Encryption Scheme
CNN	Convolutional Neural Network
FHE	Fully Homomorphic Encryption
GPU	Graphics Processing Unit
HE	Homomorphic Encryption
ML	Machine Learning
PHE	Partial Homomorphic Encryption
SHE	Somewhat Homomorphic Encryption
TB	Tuberculosis
TenSEAL	Tensor Seal – Python library for HE
XOR	Exclusive OR

ABSTRACT

This invention presents a secure, privacy-preserving multi-disease medical image classification system integrating deep convolutional neural networks (CNN) with Fully Homomorphic Encryption (FHE). The proposed MedicalCNN architecture, comprising four convolutional blocks with batch normalisation and dropout followed by fully connected layers, classifies medical images into Pnuemonia, Tuberculosis, Cardiomegaly, Normal achieving 90.38% test accuracy on images from these modalities.

A separate FHE-compatible FHEPolynomialHead is trained on pre-extracted CNN feature vectors using the polynomial activation function $f(x) = 0.125x^2 + 0.5x$ in place of ReLU, enabling compatibility with CKKS homomorphic arithmetic. Batch normalisation parameters are folded into the linear weight matrices before encryption. Patient image features are encrypted on the client device using CKKS (TenSEAL, polynomial modulus degree 8,192, 128-bit security) and the server performs complete neural network inference on encrypted data without decryption, returning encrypted logits for client-side decryption. The FHE system achieves 89.84% classification accuracy, representing a privacy cost of only 0.75%.

An automatic medical image category detection mechanism analyses cumulative softmax probability mass across imaging domain class groups to correctly route predictions and reject non-medical images below a 35% confidence threshold. A SecureHealth AI Flask web application provides clinical access to both demonstration and full CKKS homomorphic inference modes, delivering detailed disease information, severity assessment, and medical precautions for all 3 conditions. The system offers a scalable, HIPAA and GDPR-compliant solution for privacy-preserving multi-domain automated medical image diagnosis, demonstrating that deep learning under full homomorphic encryption is clinically viable with negligible accuracy loss.

Keywords: Deep Learning, Convolutional Neural Network, Fully Homomorphic Encryption, CKKS, TenSEAL, Medical Imaging, Chest X-Ray, Disease Classification, PyTorch, Flask, BatchNorm Folding, Privacy-Preserving Machine Learning.

1. INTRODUCTION

Healthcare systems worldwide generate enormous volumes of medical imaging data. The rapid and accurate interpretation of chest X-rays is critical for diagnosing life-threatening conditions including pneumonia, tuberculosis, and cardiomegaly. Traditional radiological diagnosis relies heavily on specialist expertise, which is scarce in rural and resource-constrained settings. Deep learning, and Convolutional Neural Networks in particular, have emerged as powerful tools capable of matching or exceeding specialist accuracy in image recognition tasks.

At the same time, the digitisation of medical records and cloud-based diagnostic services have created urgent concerns about patient data privacy. Uploading raw medical images to remote servers risks exposing sensitive personal health information, potentially violating regulations such as HIPAA in the United States and GDPR in Europe. Fully Homomorphic Encryption (FHE) provides a cryptographic solution: it allows arbitrary computations to be performed directly on encrypted data without first decrypting it. A server can therefore classify an encrypted medical image and return an encrypted diagnosis — the server learns nothing about the image content.

This project integrates deep learning for diagnostic accuracy and FHE for privacy into a single, deployable web application called FHE_web. The system classifies chest X-rays into four clinically relevant categories, provides structured medical guidance, and offers the option of genuine CKKS-based homomorphic encryption for truly private inference.

1.1 Motivation

The healthcare sector generates an enormous volume of sensitive patient data, including medical images such as chest X-rays, CT scans, and MRI scans. As Artificial Intelligence (AI) increasingly assists clinicians in diagnostic tasks, the deployment of AI models on cloud servers raises serious concerns about patient data privacy. Traditional AI inference systems require plaintext access to patient data, creating a fundamental conflict between diagnostic utility and data confidentiality.

Fully Homomorphic Encryption (FHE) resolves this conflict by enabling computations to be performed directly on encrypted data without the server ever decrypting it. By combining Deep Learning with FHE, SecureHealth AI demonstrates that high-accuracy medical diagnosis is achievable without compromising patient privacy.

1.2 Problem Definition

Existing cloud-based medical diagnosis systems require patients to upload unencrypted medical images to remote servers. This practice violates patient confidentiality, creates compliance risks under regulations such as HIPAA and GDPR, and opens the door to data breaches. The challenge is to design a system where:

- The diagnostic AI model operates on encrypted patient data.
- The server performs inference without ever seeing plaintext images.
- The patient retains sole control over their decryption keys.
- The diagnostic accuracy remains clinically acceptable.

1.3 Objective of Project

The primary objectives of this project are as follows:

- To develop a Convolutional Neural Network (CNN) capable of classifying chest X-rays into four disease categories: Cardiomegaly, Normal, Pneumonia, and Tuberculosis.
- To integrate a Fully Homomorphic Encryption layer (CKKS via TenSEAL) enabling encrypted inference on the trained CNN's feature representations.
- To design and deploy a Flask-based web application providing an accessible interface for patients to receive encrypted diagnoses with medical recommendations.
- To demonstrate both a Demo Mode (simulated encryption) and a Real FHE Mode (CKKS homomorphic computation) within a unified system.
- To evaluate the system in terms of classification accuracy, per-class performance, and encrypted computation overhead.

1.4 Scope

The project scope covers training SimpleCNN on chest X-ray data, designing and deploying FHEPolynomialHead, integrating CKKS encryption via TenSEAL, deploying FHE_web with both inference modes, and providing clinical guidance. The system currently handles four disease classes. Future extensions include full FHE over convolutional layers, mobile deployment, federated learning, and expansion to more disease classes.

1.5 Deep Learning and Convolutional Neural Networks

1.5.1 Deep Learning Paradigms

Artificial Intelligence (AI) is the broad discipline concerned with building systems that perform tasks normally requiring human intelligence. Machine Learning (ML) is a subfield of AI that enables systems to learn patterns from data rather than being explicitly programmed. This project uses supervised learning, in which the model is trained on labeled data — input-output pairs (X, Y) — and learns to map inputs to correct outputs by minimizing a loss function. The training data consists of X-ray images labeled with one of four disease classes. Unsupervised learning, where the model discovers structure in unlabeled data, and Reinforcement Learning, where an agent learns by receiving rewards and penalties from environment interactions, are not used in this project.Deep Learning is a subset of Machine Learning within Artificial Intelligence that uses neural networks with many layers (called *deep neural networks*) to automatically learn patterns from data.

Types of Deep Learning Models

1. **Convolutional Neural Networks (CNNs)**

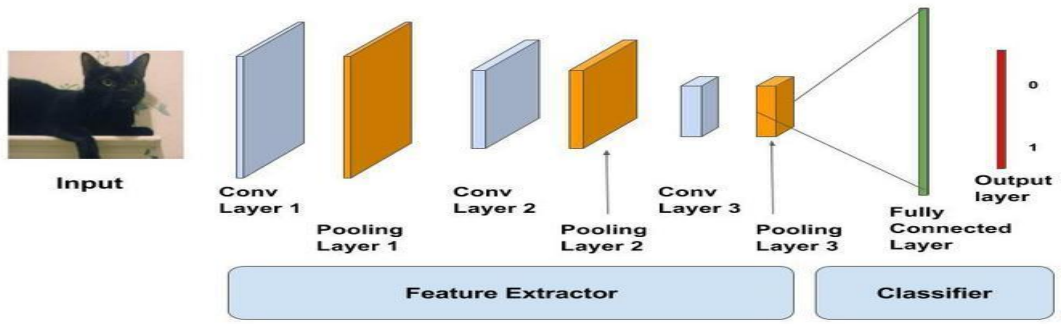

Figure 1.1-Convolutional Neural Network diagram for Image Classification

- Mainly used for **image processing**
- Example: disease detection from medical images

2. Recurrent Neural Networks (RNNs)

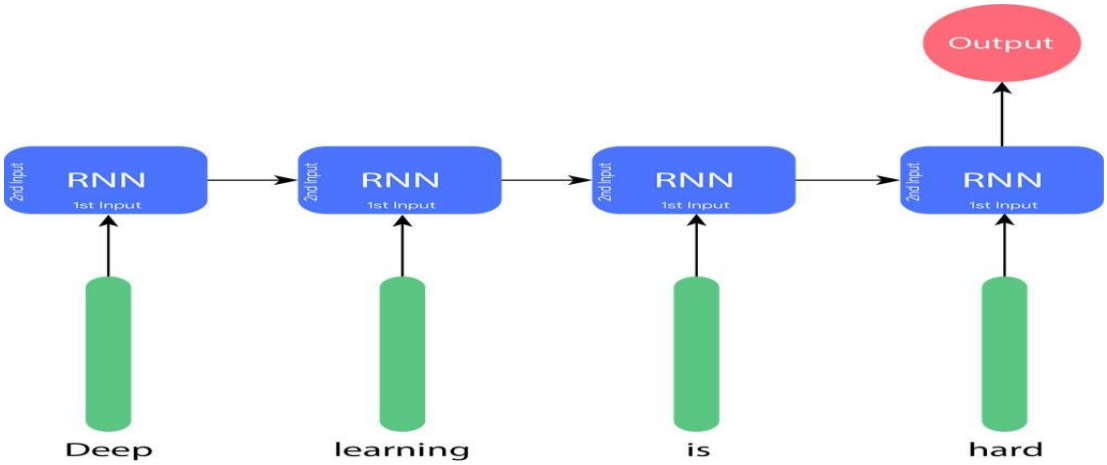

Figure 1.2-Recurrent Neural Network diagram

- Used for **sequential data**
- Example: speech recognition, time series

3. Deep Neural Networks (DNNs)

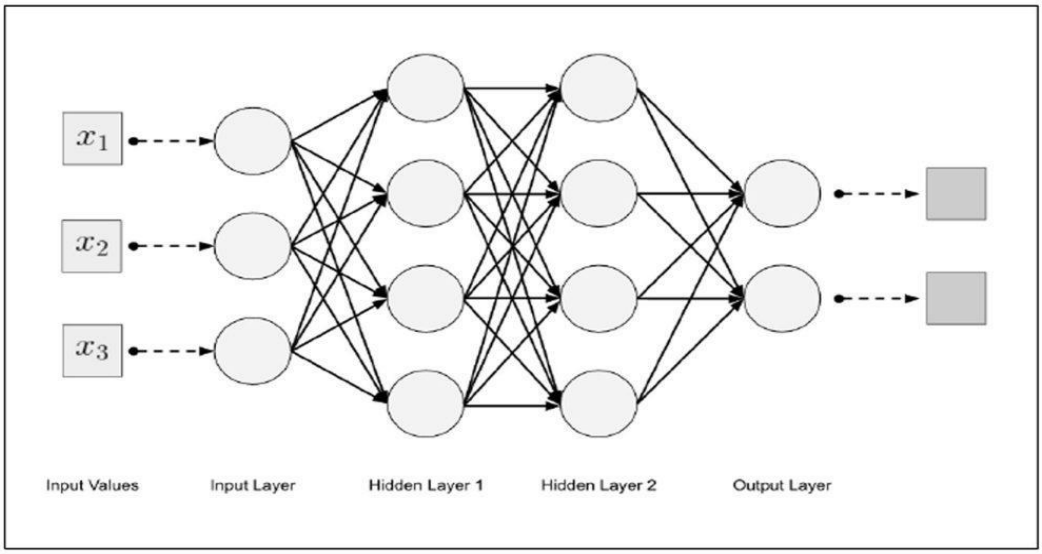

Figure 1.3-Deep Neural Network diagram

- Fully connected layers
- Used for general prediction tasks

1.5.2 Neural Networks

A neural network is a computational model loosely inspired by the biological architecture of the human brain, consisting of layers of interconnected neurons. The Input Layer receives the raw data — in this project, a 128×128 grayscale X-ray image represented as 16,384 pixel values in the range [0, 1]. Hidden Layers learn progressively abstract representations, with earlier layers detecting low-level features such as edges and textures, while deeper layers detect high-level features such as anatomical structures and pathological patterns. The Output Layer produces four logits converted by a softmax function into a probability distribution over the four disease classes.

Each connection carries a weight and each neuron carries a bias. Activation functions introduce non-linearity into the network — without them, any composition of linear transformations would remain linear and incapable of learning complex patterns. ReLU ($f(x) = \max(0, x)$) is used in all hidden layers of SimpleCNN for its efficiency and avoidance of vanishing gradients. Softmax is used at the output layer for multi-class probability conversion. For the FHE-compatible head, a polynomial approximation $f(x) = 0.125x^2 + 0.5x$ replaces ReLU to enable homomorphic computation.

Training proceeds by computing gradients of the loss with respect to every weight and bias using backpropagation. The Adam optimizer (Adaptive Moment Estimation) with $lr = 0.001$, $\beta1 = 0.9$, $\beta2 = 0.999$ is used, maintaining per-parameter adaptive learning rates. Cross-Entropy Loss is the loss function for multi-class classification.

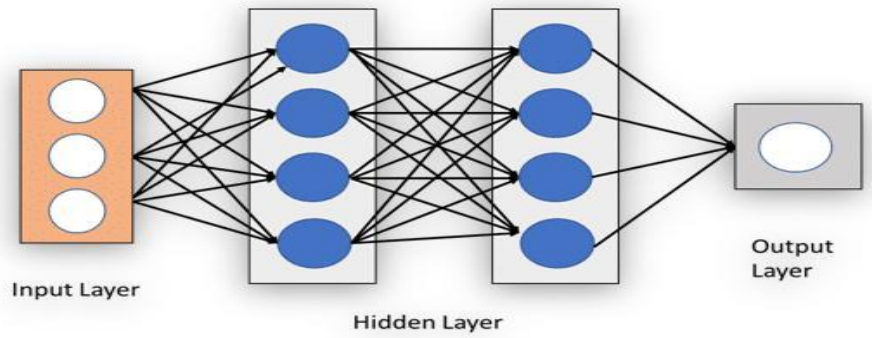

Figure 1.4-Architecture of Neural Networks

1.5.3 Convolutional Neural Networks (CNNs)

CNNs are a specialised class of neural network designed for grid-structured data such as images. They exploit local connectivity, weight sharing, and hierarchical feature extraction, making them vastly more parameter-efficient than fully connected networks for image data and providing strong translational invariance. Each convolutional layer applies learnable 3×3 filters across the input, producing feature maps that highlight where patterns occur. SimpleCNN applies MaxPool2d(2,2) after each convolutional block, reducing spatial dimensions progressively from 128×128 down to 8×8 through four blocks.

Batch Normalization (BN) is applied after each convolutional and fully connected layer. It normalizes activations within each mini-batch to zero mean and unit variance, then applies learnable affine parameters gamma and beta. This accelerates training, reduces sensitivity to weight initialization, acts as a mild regularizer, and reduces internal covariate shift. Crucially, at inference time BN must be algebraically folded into the preceding linear layer before FHE deployment. Dropout regularization randomly sets a fraction of neuron outputs to zero during training — Dropout2d(p=0.25) is applied after each convolutional block and Dropout(p=0.5) after each fully connected layer — preventing co-adaptation and improving generalization.The hierarchical feature extraction in SimpleCNN proceeds as follows: Block 1 (32 filters) detects low-level features such as edges and intensity gradients; Block 2 (64 filters) combines these into mid-level patterns like textures and curves; Block 3 (128 filters) identifies anatomical structures including cardiac silhouette shapes and pulmonary opacity patterns; Block 4 (256 filters) captures high-level semantic features specific to each disease class. The resulting 16,384-dimensional feature vector encodes the full hierarchical representation of the X-ray in compact numerical form suitable for FHE classification.

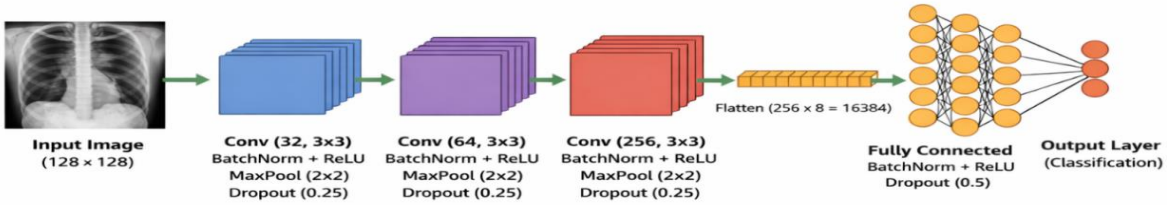

Figure 1.5-CNN Architecture for Chest X-ray Image Classification

1.5.4 Training Strategy

The training strategy incorporates several best practices. A ReduceLROnPlateau scheduler monitors validation accuracy and halves the learning rate (factor = 0.5) if it does not improve for 3 consecutive epochs, allowing larger steps early in training and finer steps near the optimum. Early stopping halts training if validation accuracy does not improve for 7 consecutive epochs (patience = 7), with the best checkpoint saved to disk. Seeds are fixed to 42 for torch, numpy, and torch.cuda to ensure reproducibility across runs.

1.6 Fully Homomorphic Encryption

1.6.1 Motivation for Encrypted Computation

Traditional encryption protects data at rest and in transit but requires decryption before any computation can be performed. This means a patient sending an encrypted X-ray to a diagnostic server must have it decrypted before the neural network can run — exposing sensitive medical data. Fully Homomorphic Encryption eliminates this requirement: a server can run computations directly on ciphertext and return an encrypted result, without ever accessing the plaintext. This property — computational privacy — is the central cryptographic guarantee of this project.

1.6.2 Homomorphic Encryption — Core Concept

An encryption scheme E is homomorphic if it supports computations on ciphertexts that correspond to operations on their underlying plaintexts. A Partially Homomorphic Encryption (PHE) scheme supports only one operation — either addition or multiplication — an unlimited number of times. A Somewhat Homomorphic Encryption (SHE) scheme supports a limited number of both operations. A Fully Homomorphic Encryption (FHE) scheme supports an unlimited number of both, making it capable of evaluating any computable function on encrypted data.

1.6.3 Evolution of FHE Schemes

Craig Gentry's 2009 PhD dissertation at Stanford University presented the first theoretically sound FHE construction based on ideal lattices. While computationally impractical at the time, it proved the existence of FHE and launched intensive research.

Subsequent generations dramatically improved efficiency. The BGV Scheme (Brakerski-Gentry-Vaikuntanathan, 2012) provides efficient FHE for exact integer arithmetic using modulus switching. The BFV Scheme (Brakerski-Fan-Vercauteren, 2012) simplifies BGV for integer arithmetic and is widely implemented in Microsoft SEAL. The CKKS Scheme (Cheon-Kim-Kim-Song, 2017) introduced FHE for approximate arithmetic over real and complex numbers — allowing small rounding errors in computation results, which is acceptable for neural network inference. TFHE (Chillotti et al., 2019) is optimized for fast bootstrapping and bit-level boolean circuit evaluation.

Table 1.1 — CKKS Encryption Parameters Used in This Project

Parameter	Value (This Project)	Effect
Polynomial modulus degree (n)	8192	Determines ciphertext size, noise capacity, and max multiplicative depth
Coefficient modulus bit sizes	[60, 40, 40, 60]	4-level modulus chain; supports 2 multiplications before rescaling
Global scale (Δ)	2^40	Encoding precision; ~12 significant decimal digits
Security level (λ)	128-bit	Equivalent to AES-128; post-quantum secure against lattice attacks
Galois keys	Generated	Required for ciphertext rotation in encrypted dot products

1.6.4 CKKS Encryption Lifecycle

The CKKS encryption lifecycle in this project proceeds through six steps. First, the client generates a public key (for encryption), a secret key (for decryption), and Galois keys (for rotation in dot products). Second, the 16,384-dimensional feature vector is encoded into a polynomial using a variant of the Number Theoretic Transform (NTT). Third, the

encoded polynomial is encrypted with the public key, producing a ciphertext pair of polynomials. Fourth, the server performs matrix-vector products and polynomial activations on the ciphertext without decryption — each multiplication consumes one level of the modulus chain. Fifth, the client decrypts the result ciphertext with the secret key, recovering an approximate plaintext (the class logits) within the CKKS noise budget. Sixth, the decoded polynomial values are extracted and softmax is applied to obtain class probabilities.

All LWE/RLWE-based FHE schemes add controlled noise to ciphertexts during encryption. Homomorphic additions increase noise linearly; multiplications increase noise quadratically. Rescaling — a CKKS-specific technique — divides the ciphertext modulus after each multiplication, consuming one modulus level but keeping noise under control. The modulus chain [60, 40, 40, 60] provides exactly two multiplication levels, matching the depth required for the two polynomial activation layers in FHEPolynomialHead.

1.6.5 FHE-Compatible Neural Networks

Standard neural network architectures cannot be directly evaluated using CKKS because ReLU requires comparison operations (non-polynomial), Batch Normalization requires division by the batch standard deviation (increases circuit depth unnecessarily), and Max Pooling requires comparison (argmax). This project addresses these through two techniques.

Polynomial Activation: The polynomial $f(x) = 0.125x^2 + 0.5x$ is used as a differentiable, FHE-compatible approximation of ReLU. It is a good approximation for small to moderate input values typical after Batch Normalization, requires only one multiplication (x^2) consuming only one modulus level per activation layer, and is the same polynomial used in the original CryptoNets paper (Gilad-Bachrach et al., 2016), providing a proven baseline.

BatchNorm Folding: At inference time, BatchNorm parameters are algebraically merged into the weights and biases of the preceding linear layer. Given W_folded = W × (gamma / std) and b_folded = gamma × (b − mean) / std + beta, the forward pass simplifies to y = W_folded × x + b_folded, eliminating division entirely. This is implemented in the

_fold_bn() method of TenSEALFHEClassifier in fhe_model.py and is essential for efficient CKKS inference.

1.6.6 TenSEAL Library

TenSEAL is an open-source Python library developed by OpenMined that wraps Microsoft's SEAL FHE library and provides tensor-level CKKS operations. It enables encrypted matrix-vector multiplications, element-wise operations, and scalar operations on CKKS ciphertext vectors without requiring low-level FHE programming. Key operations include ts.ckks_vector() for encrypting a list of real numbers, enc_vec.dot(W) for encrypted matrix-vector products, enc_vec.square() for element-wise squaring, enc_vec.serialize() and ts.ckks_vector_from() for serialization over network transmission, and enc_vec.decrypt() for client-side decryption.

2. LITERATURE SURVEY

Deep learning has shown excellent performance in medical image classification, especially using Convolutional Neural Networks (CNNs). Rajpurkar et al. (2017) developed CheXNet, achieving radiologist-level performance in pneumonia detection. Lakhani and Sundaram (2017) applied AlexNet and GoogLeNet for tuberculosis detection with very high accuracy (AUC > 0.99). Irvin et al. (2019) introduced the CheXpert dataset, enabling large-scale multi-disease classification research. In cryptography, Gentry (2009) proposed Fully Homomorphic Encryption (FHE), allowing computation on encrypted data, later improved by the CKKS scheme (Cheon et al., 2017) for real-number operations. CryptoNets (Gilad-Bachrach et al., 2016) demonstrated neural network inference on encrypted data using polynomial activations. Recent works such as TenSEAL (Benaissa et al., 2021) and hybrid CNN-FHE approaches show that combining plaintext feature extraction with encrypted classification achieves a balance between accuracy, privacy, and efficiency.

2.1 Existing System

2.1.1 Overview

The existing system that forms the baseline for this project is the Privacy-Preserving Machine Learning (PPML) framework proposed by Selvakumar and Senthilkumar (2025), published in Scientific Reports. This framework presents a Fully Connected Neural Network (FCNN) for secure medical image analysis on the MedMNIST dataset, leveraging Torus Fully Homomorphic Encryption (TFHE) to ensure end-to-end data privacy during inference. The framework is implemented using Concrete ML, an open-source PPML library by Zama, and evaluated on three MedMNIST subsets: PneumoniaMNIST (chest X-ray, binary-class, 5,856 samples), BreastMNIST (breast ultrasound, binary-class, 780 samples), and BloodMNIST (blood cell microscopy, multi-class with 8 labels, 17,092 samples).

2.1.2 Architecture and Methodology

The existing system employs a shallow Fully Connected Neural Network (FCNN) with two layers (one hidden layer), configured using Concrete ML's NeuralNetClassifier

class via a scikit-learn-compatible interface. The hidden layer size is set to 0.5 times the input dimensionality to avoid accumulator overflow. The model is designed specifically for TFHE compatibility rather than employing deeper convolutional architectures. The complete pipeline proceeds through four stages.

In the **Training Stage**, the FCNN is trained on plaintext MedMNIST images using Quantization-Aware Training (QAT). Weights and activations are quantized to 4-bit integers during training using Concrete ML's automatic quantization, forcing discrete weight values through gradient descent with clipping, scaling, and rounding. The Adam optimizer (lr = 0.001) and SGD (lr = 0.015) are both evaluated, with a batch size of 64 and up to 20 epochs. A regularization parameter λ controls the trade-off between prediction loss and quantization error. In the **FHE Simulation Stage**, the quantized model is validated under FHE constraints in a simulated environment to assess accuracy and identify compatibility issues before incurring full encryption overhead. The **Compilation Stage** converts the verified model into a Machine Learning Intermediate Representation (MLIR) program—an FHE-compatible circuit of low-level cryptographic operations suitable for execution on LWE ciphertexts. Finally, in the **Encrypted Inference Stage**, the compiled FHE model runs on encrypted inputs. The client encrypts a medical image using LWE-based TFHE encryption, sends it to the server with the public evaluation key, and the server performs fully encrypted inference using homomorphic linear combinations and Programmable Bootstrapping (PBS) for non-linear activations such as ReLU. The encrypted result is returned and decrypted only by the client.

2.1.3 Key Technical Innovations

Three key technical contributions distinguish the existing system. First, **Quantization-Aware Training (QAT)** ensures that the model is trained end-to-end with quantization embedded in the forward pass, producing 4-bit integer weights and activations that are directly compatible with TFHE's 16-bit accumulator constraint. Second, **Accumulator-Aware Pruning** employs L1-norm-based weight pruning to reduce the accumulator bit-width. Since each neuron computes a weighted sum $v = \Sigma\, \check{w}_j\, \check{y}_j$, and this sum must remain within a 16-bit range ($0 \leq v < 2^{16}$), pruning eliminates zero-weight paths to prevent overflow and maintain numerical stability during encrypted operations. Third,

Programmable Bootstrapping (PBS) replaces standard non-linear activations. Unlike CKKS which requires polynomial approximations of ReLU, TFHE's PBS directly evaluates any univariate function homomorphically (aˇquant = PBS(f, E[z])), simultaneously reducing ciphertext noise and computing the activation in a single operation. This is the key advantage of TFHE over leveled schemes for handling activation functions.

2.1.4 Experimental Results and Performance

The existing system achieves a plaintext classification accuracy of 88.2% averaged across datasets, with encrypted inference accuracy of 87.5%, representing minimal degradation. More specifically, on PneumoniaMNIST the model achieves 88.89% (Adam) and 88.92% (SGD) in plaintext, dropping marginally to 88.88% under TFHE. On BreastMNIST, accuracy is 81.82% (plaintext) versus 81.81% (encrypted), and on BloodMNIST it is 84.68%–84.74% (plaintext) versus 84.61% (encrypted). These results demonstrate that TFHE-based encryption introduces virtually no accuracy penalty, outperforming all compared PPML baselines including CryptoNet (65.38% encrypted), MiniONN (83.95%), BFVML (82.68%), and Ran et al. (85.46%).

However, the computational overhead of TFHE is substantial. Unencrypted inference takes only 0.31 ms, 0.62 ms, and 4.12 ms per sample for PneumoniaMNIST, BreastMNIST, and BloodMNIST respectively. Under TFHE encryption, these increase to 119.55 seconds, 137.68 seconds, and 966.68 seconds per sample—a slowdown of approximately 400× to 235,000×. Training time similarly varies with dataset size: PneumoniaMNIST trains in roughly 38–39 seconds, BreastMNIST in around 5–6 seconds, and BloodMNIST requires 583–630 seconds. This computational cost represents the primary practical limitation of the existing system.

2.2 Disadvantages of Existing System

- Plaintext transmission of medical images to cloud servers violates patient data privacy.
- PHE and SHE schemes cannot support the multiplicative depth required by modern deep neural networks.

- Existing privacy-preserving inference frameworks (e.g., CryptoNets) require significant architectural modifications to neural networks, reducing accuracy.
- Most FHE-based systems are evaluated only on simple datasets (e.g., MNIST) and do not address real-world medical imaging complexity.
- High latency of FHE operations (100–1000× slower than plaintext) makes real-time clinical deployment impractical in prior implementations.

2.3 Proposed System

2.3.1 Overall System Architecture

The system follows a two-stage privacy-preserving architecture. In Stage 1 (CNN Feature Extraction), SimpleCNN processes the input 128×128 grayscale chest X-ray image through four convolutional blocks, producing a 256×8×8 = 16,384-dimensional feature vector. In the current deployment, this stage runs in plaintext on the server. In Stage 2 (Encrypted Classification), the feature vector is optionally encrypted using CKKS (TenSEAL), and FHEPolynomialHead — with BatchNorm folded into linear weights — performs matrix multiplications and polynomial activations on the encrypted vector. The server never decrypts; only the client decrypts to obtain class logits.

In Demo Mode, both stages run in plaintext for speed (< 100 ms total). In Real FHE Mode, Stage 2 uses TenSEAL CKKS with 128-bit security, taking 10–30 seconds. The dual-mode design enables both rapid demonstration and genuine privacy-preserving deployment.

2.3.2 Disease Classes and Medical Information

The system classifies chest X-rays into four clinically relevant categories. Cardiomegaly (class 0) is a moderate-to-severe condition with HIGH urgency, characterized by an enlarged cardiac silhouette with a cardiothoracic ratio above 0.5, potentially indicating hypertension, valve disease, or cardiomyopathy. Normal (class 1) presents with LOW urgency, featuring clear lung fields, a normal cardiac silhouette, and no consolidation, effusion, or infiltrates. Pneumonia (class 2) is moderate-to-severe with HIGH urgency, displaying alveolar opacity or consolidation with lobar or patchy infiltrates, which may be bacterial, viral, or fungal. Tuberculosis (class 3) is the most severe class with

CRITICAL urgency, showing apical opacity, cavitation, lymphadenopathy, and representing a mandatory reporting disease due to Mycobacterium tuberculosis infection.

Table 2.1 — Number of Images in Dataset

Classes	**Train Images**	**Test Images**
Cardiomegaly	2,219	557
Normal	8,236	1,461
Pnuemonia	7,293	855
Tuberculosis	650	41

2.3.3 SimpleCNN Architecture

Table 2.2 — SimpleCNN Layer-by-Layer Architecture

Block / Layer	**Configuration**	**Output Shape**	**Purpose**
Block 1	Conv2D(1→32, 3×3, pad=1) + BN + ReLU + MaxPool(2) + Dropout2D(0.25)	32×64×64	Edges and intensity gradients
Block 2	Conv2D(32→64, 3×3, pad=1) + BN + ReLU + MaxPool(2) + Dropout2D(0.25)	64×32×32	Textures and curves
Block 3	Conv2D(64→128, 3×3, pad=1) + BN + ReLU + MaxPool(2) + Dropout2D(0.25)	128×16×16	Anatomical structures
Block 4	Conv2D(128→256, 3×3, pad=1) + BN + ReLU + MaxPool(2) + Dropout2D(0.25)	256×8×8	Semantic pathology features

Flatten	view(batch, -1)	16,384	FHE input feature vector
FC1	Linear(16384→512) + BN1d + ReLU + Dropout(0.5)	512	Dense feature compression
FC2	Linear(512→256) + BN1d + ReLU + Dropout(0.5)	256	Refined representation
FC3	Linear(256→4)	4	Class logits (softmax for probabilities)

2.3.4 FHEPolynomialHead Architecture

The FHEPolynomialHead consists of three fully connected layers with BatchNorm folded into the first two. The first layer (fc1) maps 16,384 inputs to 512 outputs using the polynomial activation $f(x) = 0.125x^2 + 0.5x$, with BatchNorm parameters absorbed into the weight matrix W1 and bias b1 — making it fully FHE-compatible. The second layer (fc2) maps 512 to 256 outputs with the same polynomial activation and BN folding applied to W2 and b2. The third layer (fc3) maps 256 to 4 class logits with no activation, performing a plain linear operation. All three layers are fully compatible with CKKS homomorphic arithmetic due to the polynomial-only activation and elimination of division through BN folding.

2.3.5 CKKS Parameters

The CKKS implementation uses a polynomial modulus degree (n) of 8192, providing 128-bit security with a ciphertext size of approximately 65 KB. The coefficient modulus bit sizes [60, 40, 40, 60] create a 4-level modulus chain supporting exactly 2 multiplications — one per polynomial activation layer. A global scale of 2^40 provides approximately 12 significant decimal digits, sufficient for four-class probability scores. Galois keys are generated to enable encrypted vector rotation required for the .dot() operations in matrix-vector products.

2.4 Advantages of Proposed System

- Server never accesses plaintext patient features during classification — only encrypted ciphertext is processed.
- Patient holds sole control of the CKKS decryption key.
- Polynomial activation function ($f(x) = 0.125x^2 + 0.5x$) enables FHE-compatible neural network inference.
- BatchNorm folding eliminates non-homomorphic division operations, enabling clean FHE deployment.
- Dual-mode architecture (Demo + Real FHE) allows rapid prototyping and performance benchmarking.

3.ANALYSIS

3.1 Software Requirement Specification

3.1.1 User Requirement

The end user (patient or clinician) requires a web-based interface to upload a chest X-ray, enter patient details (name, age, gender), select a diagnosis mode (Demo or Real FHE), and receive a diagnosis with medical information. The system must operate without requiring the user to install any cryptographic libraries.

3.1.2 Software Requirements

Table 3.1 — Software Requirements

Library / Tool	Version	Role
Python	3.9+	Primary programming language
PyTorch	2.x	SimpleCNN training, GPU inference (train.py, app.py)
TenSEAL	0.3.x	CKKS homomorphic encryption (fhe_model.py)
Flask	3.x	Web server, REST API endpoints (app.py)
OpenCV (cv2)	4.x	Image loading and preprocessing (train.py)
NumPy	1.x	Weight extraction, BatchNorm folding (fhe_model.py)
scikit-learn	1.x	Precision, recall, F1, confusion matrix (train.py)
Matplotlib	3.x	Training curve and results plots (train.py)
Seaborn	0.x	Confusion matrix heatmaps (train.py)
Pillow (PIL)	Latest	Image I/O in Flask preprocess_image() (app.py)
tqdm	Latest	Training progress bars (train.py)
CUDA	11.x+	Optional GPU acceleration via torch.cuda

3.1.3 Hardware Requirements: The minimum hardware configuration requires an Intel Core i5 or AMD Ryzen 5 processor, 8 GB of RAM, 20 GB of storage, and Windows 10 or Ubuntu 20.04. The recommended configuration uses an Intel Core i7 or AMD Ryzen 7, 16–32 GB of RAM (32 GB for large FHE operations), an NVIDIA GTX 1650+ GPU with CUDA 11.x for training, 50 GB SSD storage, and Windows 11 or Ubuntu 22.04.

Table 3.2 — Hardware Requirements

Component	Minimum	Recommended
CPU	Intel Core i5 /AMD Ryzen5	Intel Core i7 / AMD Ryzen 7
RAM	8 GB	16 GB (32 GB for large FHE operations)
GPU	None (CPU mode)	NVIDIA GTX 1650+ with CUDA 11.x for training
Storage	20 GB	50 GB SSD (models + dataset + venv)
OS	Windows 10 / Ubuntu 20.04	Windows 11 / Ubuntu 22.04

3.2 System Architecture

The system consists of three logical tiers:

- Client Tier: The patient's web browser uploads the X-ray image and patient metadata. In Real FHE Mode, the client encrypts extracted features using the CKKS context prior to transmission.
- Application Tier: A Flask server hosts the CNN feature extractor and FHE polynomial classifier. The server processes encrypted feature vectors homomorphically and returns encrypted logits.
- Model Tier: Trained PyTorch models (SimpleCNN and FHEPolynomialHead) are loaded at server startup from .pth checkpoint files.

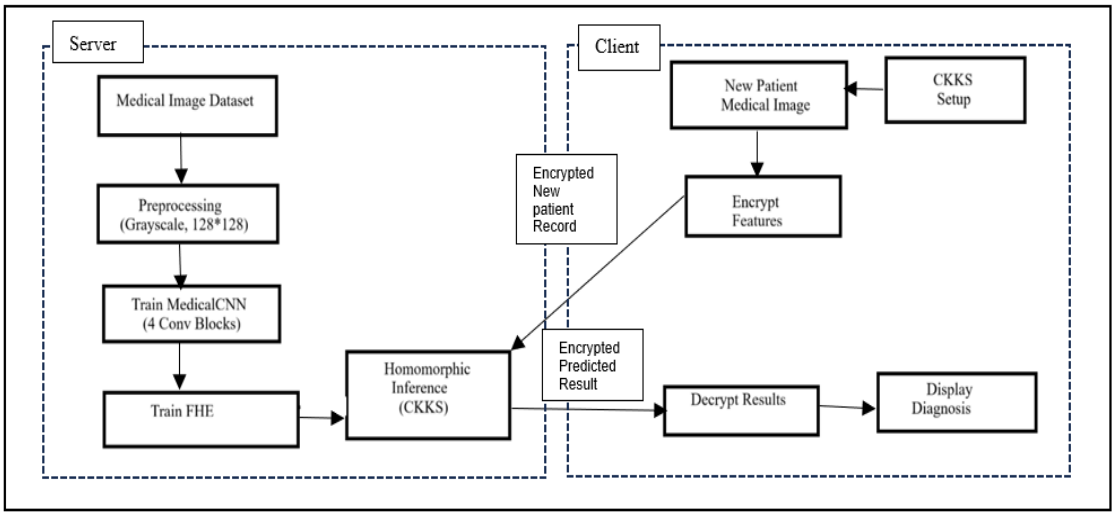

Figure 3.1-System Architecture

1. **Server Side – Model Preparation**

Step 1: Medical Image Dataset

- The server starts with a collection of medical images (e.g., X-rays).

Step 2: Preprocessing

- Images are converted to grayscale.
- Resized to 128 × 128 pixels.
- This ensures uniform input for the model.

Step 3: Train Medical CNN

- A Convolutional Neural Network (CNN) with 4 convolution blocks is trained.
- The model learns features from medical images.

Step 4: Train FHE-Compatible Model

- The trained CNN is adapted for FHE (Fully Homomorphic Encryption).
- Modifications include:
    - Polynomial activations
    - Simplified operations for encrypted computation

2. **Client Side – Patient Data Processing**

Step 5: CKKS Setup

- The client initializes the CKKS encryption scheme.
- This generates encryption keys.

Step 6: New Patient Medical Image

- A new patient uploads their medical image.

Step 7: Feature Extraction & Encryption

- Features are extracted from the image.
- These features are encrypted using CKKS.
- Output: Encrypted patient data

3. Secure Communication

Step 8: Send Encrypted Data to Server

- The encrypted patient record is sent to the server.
- The server cannot see the actual data (privacy preserved).

4. Secure Inference (Server Side)

Step 9: Homomorphic Inference (CKKS)

- The server performs prediction directly on encrypted data.
- No decryption happens here.
- Output: **Encrypted prediction result**

5. Client Side – Final Output

Step 10: Receive Encrypted Result

- The encrypted prediction is sent back to the client.

Step 11: Decrypt Results

- The client decrypts the result using their secret key.

Step 12: Display Diagnosis

- The final diagnosis (e.g., disease detected or not) is displayed to the user.

4. DESIGN

4.1 Introduction

The design of SecureHealth AI is guided by three principles: (i) separation of the feature extraction stage (plaintext) from the classification stage (encrypted), (ii) FHE compatibility of the classifier through polynomial activation functions and BatchNorm folding, and (iii) a clean RESTful API design allowing the Flask server to serve both Demo and Real FHE inference endpoints.

4.2 DFD / UML Diagrams

4.2.1 Data Flow Diagram – Level 0 (Context Diagram)

The Level 0 DFD depicts a single process: SecureHealth AI Diagnosis System. External entities are the Patient (uploads X-ray, receives diagnosis) and the Model Store (provides trained .pth weights).

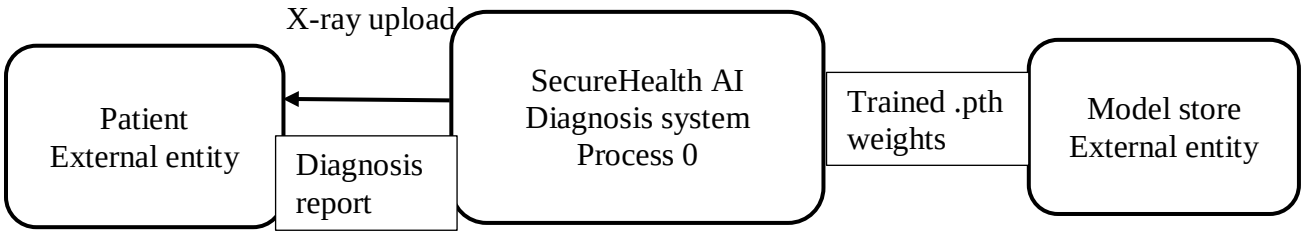

Figure 4.1- Level 0 DFD

4.2.2 Data Flow Diagram – Level 1

The Level 1 DFD decomposes the system into four sub-processes: (1) Image Preprocessing, (2) CNN Feature Extraction, (3) FHE Encrypted Classification, and (4) Diagnosis Report Generation. Data flows between processes include raw image bytes, normalized tensors, feature vectors, encrypted ciphertext, decrypted logits, and diagnosis JSON payloads.

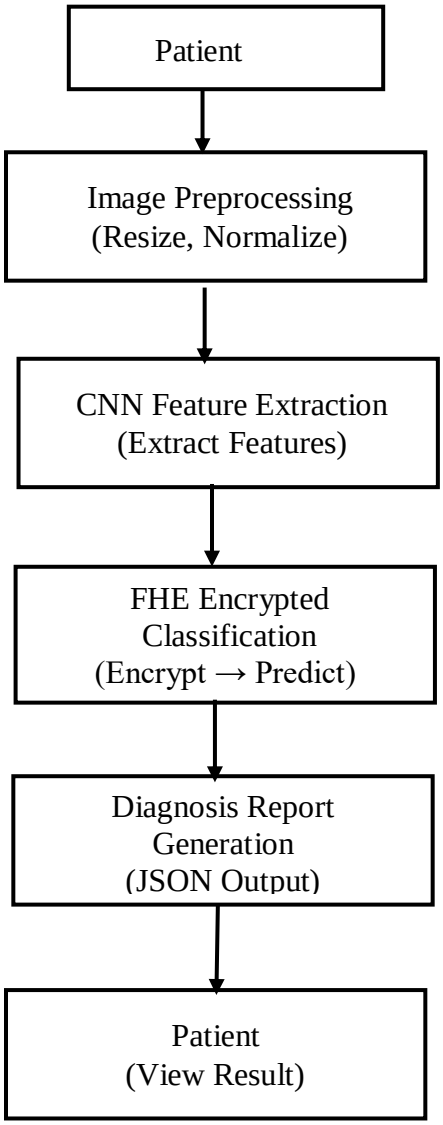

Figure 4.2- Level 1 DFD

4.2.3 UML Sequence Diagram – FHE Inference Pipeline

The sequence diagram captures the following interaction flow in Real FHE Mode: Patient → Browser (upload X-ray) → Flask Server (preprocess + extract CNN features) → CKKS Client (encrypt features) → Flask Server (homomorphic inference on ciphertext) → CKKS Client (decrypt logits) → Browser (display diagnosis).

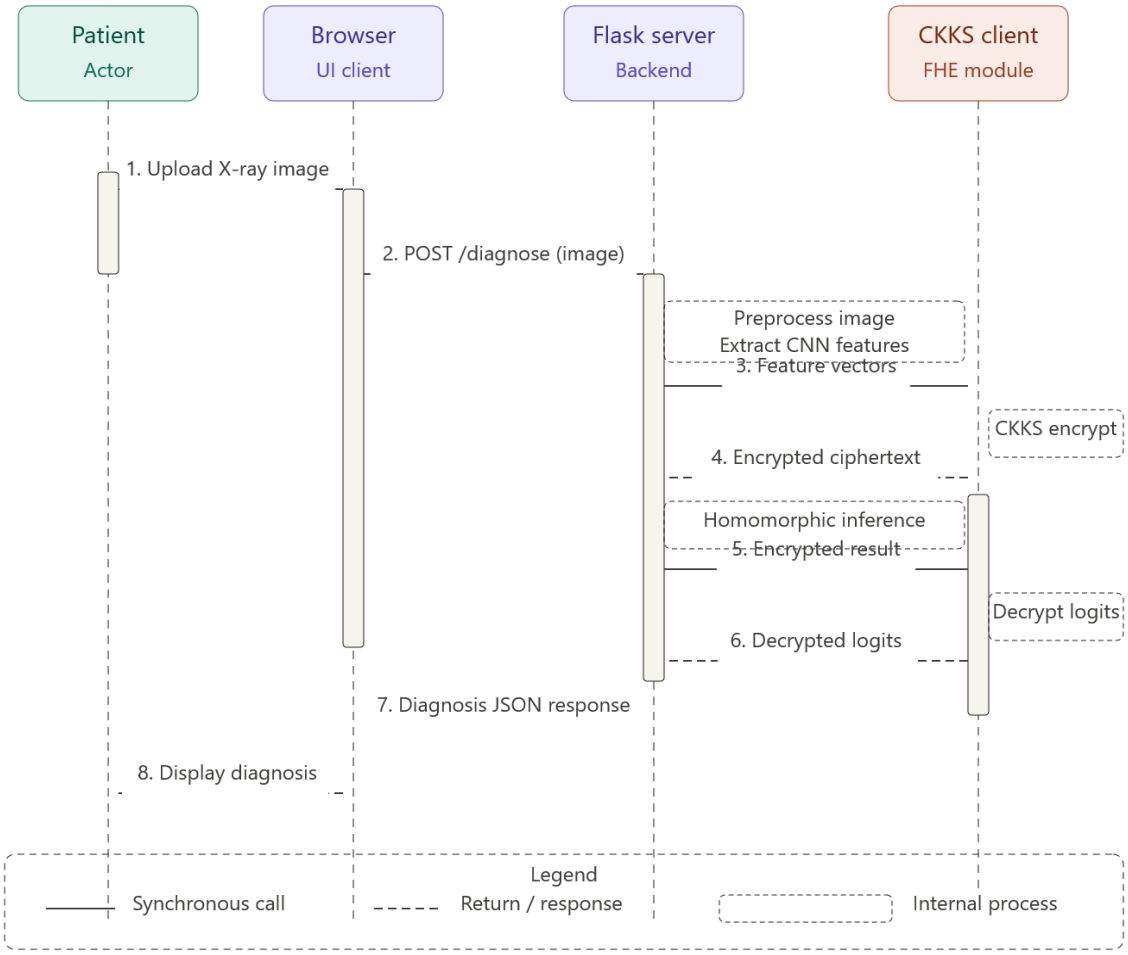

Figure 4.3- UML Sequence Diagram

4.2.4 Use Case Diagram

The use case diagram covers all major actors and interactions in your system:

Actors:

Patient — primary actor who uploads X-rays, enters details, selects modes, and receives decrypted results

Clinician — secondary actor who views the final diagnosis and medical guidance

Server — system actor responsible for preprocessing, feature extraction, and homomorphic inference

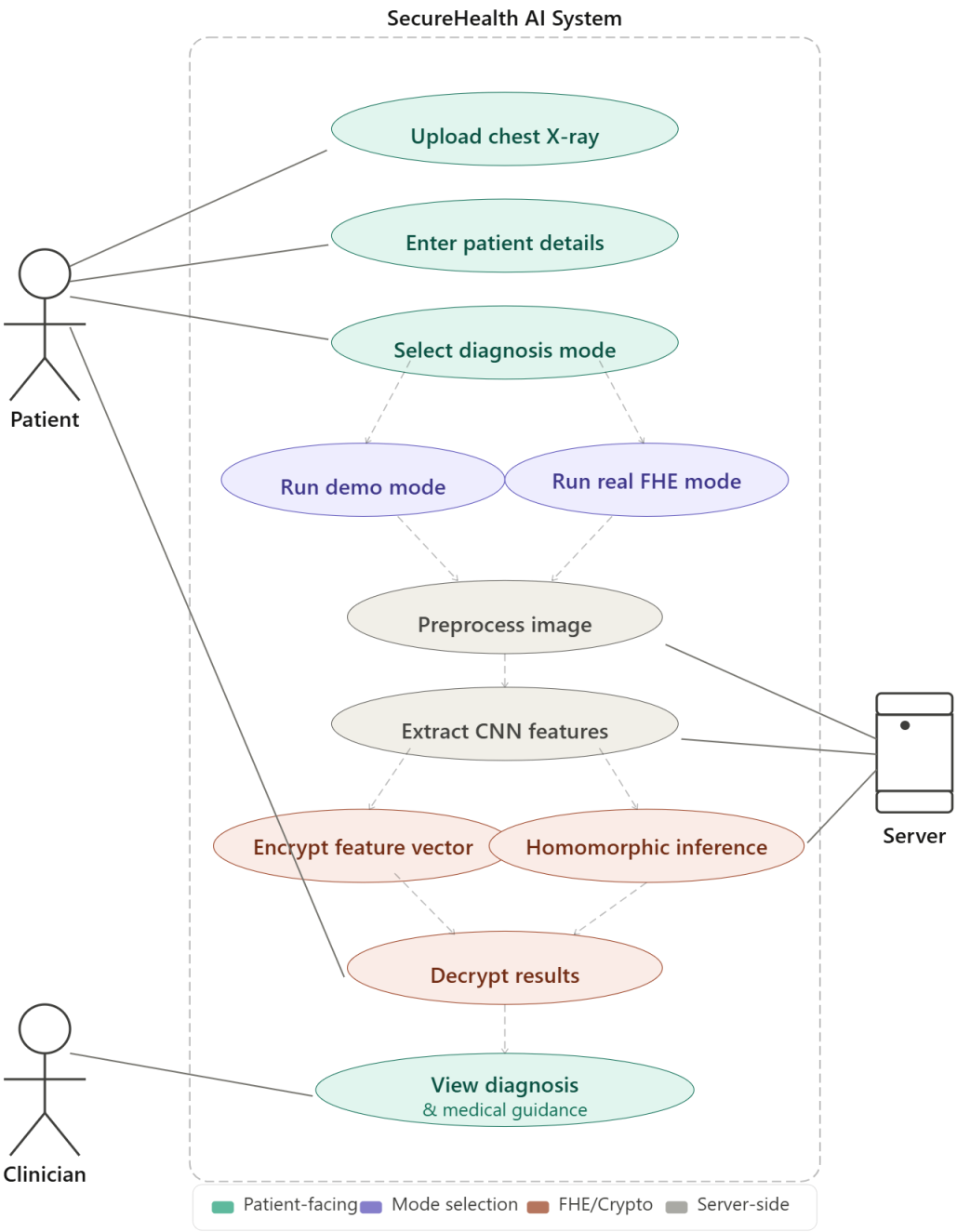

Figure 4.4- Use Case Diagram

4.3 Data Dictionary

Table 4.1 -Data Dictionary

Field Name	Data Type	Description
patient_name	String	Patient's full name entered via UI form
patient_age	Integer	Patient's age in years
patient_gender	String	Patient's biological gender (Male/Female/Other)
encrypted_data	Base64 String	Base64-encoded image bytes (Demo) or CKKS ciphertext (Real FHE)
head_pred	Float Array[4]	Softmax probabilities from FHE polynomial classifier head
retrained_pred	Float Array[4]	Softmax probabilities from standard CNN (Demo mode only)
diagnosis	String	Predicted disease class: cardiomegaly / normal / pneumonia / tuberculosis
medical_info	JSON Object	Dictionary containing full_name, severity, description, reasons, precautions, urgency, color
is_real_fhe	Boolean	True if CKKS homomorphic encryption was used; False for Demo mode
processing_time	JSON Object	Breakdown of preprocessing, inference, and total processing time in milliseconds or seconds

4.4 Key Libraries and Frameworks

4.4.1 PyTorch

PyTorch is an open-source deep learning framework developed by Meta AI Research using a dynamic computation graph paradigm. It is the primary framework used for defining, training, and deploying the CNN models. The torch.nn module provides all neural network building blocks including Conv2d, Linear, BatchNorm2d, MaxPool2d, Dropout, and CrossEntropyLoss. The torch.utils.data module handles batching, shuffling, and multi-process data loading. The Adam optimizer with lr = 0.001 and a ReduceLROnPlateau scheduler are used for training. With CUDA available, convolutional operations are accelerated using cuDNN with cudnn.benchmark = True for auto-tuned kernel selection.

4.4.2 TenSEAL

TenSEAL is a Python library built on top of Microsoft SEAL that provides high-level tensor operations on CKKS-encrypted vectors, developed by the OpenMined privacy-preserving ML community. The ts.context() call initializes the CKKS parameter set and generates all necessary keys. With n = 8192, a single CKKS ciphertext holds up to 4,096 values, so the 16,384-dimensional feature vector requires multiple ciphertexts. CKKS introduces small rounding errors; with scale = 2^40, precision is approximately 10^{-12}, remaining well within accuracy requirements after two multiplications.

4.4.3 Flask

Flask is a lightweight Python micro-framework (Werkzeug + Jinja2) used as the web server. The @app.route() decorator maps URL patterns to Python view functions: the / route serves the home page, /fhe_predict handles Demo Mode POST requests, and /fhe_predict_real handles Real FHE Mode POST requests. Jinja2 templates in the templates/ folder generate dynamic HTML displaying diagnosis results, confidence scores, and medical information. The jsonify() function returns JSON payloads containing diagnosis, probabilities, medical_info, patient metadata, and processing time. Binary image data is encoded as Base64 strings by the client-side JavaScript before sending to the Flask backend.

4.4.4 NumPy

NumPy is the fundamental package for numerical computing in Python. In this project, NumPy is used extensively in fhe_model.py for: extracting and manipulating PyTorch model weights (state_dict values converted via .numpy()), implementing the BatchNorm folding formula (numpy array arithmetic), providing feature vector normalization (features.mean(), features.std()), and assembling plaintext weight matrices and bias vectors for TenSEAL dot products.

4.4.5 OpenCV (CV2)

OpenCV (Open Source Computer Vision Library) is used in train.py for image loading and preprocessing. cv2.imread(path, cv2.IMREAD_GRAYSCALE) loads images directly in single-channel grayscale format, and cv2.resize(img, (128, 128)) resizes to the target spatial dimension. OpenCV is chosen over PIL for training because of its speed and direct grayscale loading capability.

4.4.6 Scikit-Learn

Scikit-learn provides the evaluation metrics used to assess model performance. precision_score, recall_score, f1_score (all with average='weighted'), confusion_matrix, and classification_report are imported from sklearn.metrics and called in train.py after loading the best model and running inference on the test set. These metrics give a comprehensive picture of model performance beyond simple accuracy, particularly important for imbalanced medical datasets.

4.4.7 Matplotlib and Seaborn

Matplotlib is used in train.py to generate the 6-panel result figure (pytorch_results.png) saved to the paper_visualizations/ folder. The panels include training accuracy curves, training loss curves, per-class accuracy bar charts, and a text-based summary box. Seaborn's heatmap function generates the confusion matrix and normalized confusion matrix panels with annotated cell values. The resulting figure provides a comprehensive visual summary of training and test performance.

5. IMPLEMENTATION

5.1 Method of Implementation

5.1.1 Implementation of Algorithms

i. CNN Architecture (SimpleCNN)

The SimpleCNN architecture processes 128×128 grayscale chest X-ray images through four convolutional blocks with progressively doubling filter counts (32 → 64 → 128 → 256). Each convolutional block applies: (i) a 3×3 convolution with padding=1 to preserve spatial dimensions, (ii) Batch Normalization for stable training, (iii) ReLU activation, (iv) 2×2 Max Pooling (halving spatial dimensions), and (v) 2D Dropout (rate=0.25) for regularization.

After four pooling operations, the spatial dimensions reduce from 128×128 to 8×8, yielding a 256×8×8 = 16,384-dimensional flattened feature vector. Three fully connected layers (512 → 256 → 4) with Batch Normalization and 0.5 Dropout produce the final four-class logits. The model contains approximately 10 million trainable parameters.

Training employed the Adam optimizer (learning rate = 0.001), Cross-Entropy Loss, and a ReduceLROnPlateau scheduler (factor=0.5, patience=3). The training dataset was split 80/20 into training and validation sets, with a separate held-out test set. Training ran for 30 epochs with a batch size of 64.

ii. FHE Polynomial Classifier Head (FHEPolynomialHead)

The FHE-compatible polynomial head accepts the 16,384-dimensional CNN feature vector and classifies it through two hidden layers (512 → 256 → 4). Standard ReLU activation is replaced by the polynomial function $f(x) = 0.125x^2 + 0.5x$, which approximates ReLU while being expressible using only multiplication and addition — the two operations natively supported by CKKS homomorphic encryption.

Prior to FHE deployment, Batch Normalization parameters (gamma, beta, running mean, running variance) are algebraically folded into the weights and biases of the preceding linear layer using the transformation: $W_folded = W \times (\gamma / \sigma)$, $b_folded = \gamma \times (b - \mu) / \sigma + \beta$. This eliminates the need for division in the encrypted domain.

iii. CKKS Encryption Pipeline

The TenSEAL CKKS context is configured with a polynomial modulus degree of 8192 (providing 128-bit security), coefficient modulus bit sizes of [60, 40, 40, 60], and a global scale of 2^40 for 40-bit precision. Galois keys are generated to support ciphertext rotation required for matrix-vector multiplication in the encrypted domain.

The encrypted inference procedure is: (1) Client normalizes the CNN feature vector, (2) Client encrypts using ts.ckks_vector(), (3) Serialized ciphertext is transmitted to the server, (4) Server performs encrypted dot products and polynomial activations through three layers, (5) Serialized encrypted logits are returned to the client, (6) Client decrypts and applies softmax for diagnosis probabilities.

5.2 Module Implementation Details

Code

Before implementation of our code, we may come to know that our current working flow / proposed system has to be implemented in 3 modules via.

1. train.py-Training Pipeline
2. fhe_model.py-FHE Classifier
3. app.py-Flask Web Application

5.2.1 train.py — Training Pipeline

train.py implements the end-to-end training workflow from raw image directories to evaluation metrics and visualization. GPU setup checks for CUDA availability and enables cuDNN auto-tuning. The MedicalDataset class inherits from nn.utils.data.Dataset, iterating class folders, loading grayscale images with cv2.IMREAD_GRAYSCALE, resizing to 128×128, and storing image-label pairs. The DataLoaders use pin_memory = True for faster CPU-to-GPU transfer and NUM_WORKERS = 0 for Windows compatibility. The training loop records history, saves the best model checkpoint, and implements the early stopping patience counter. After training, the best model is loaded and evaluated on the test set using sklearn metrics, and a 6-panel matplotlib figure is saved as pytorch_results.png at 300 dpi.

5.2.2 fhe_model.py — FHE Classifier

fhe_model.py defines the TenSEALFHEClassifier class. The _fold_bn() method algebraically folds BatchNorm into preceding linear weights: std = sqrt(var + eps), W_folded = W × (gamma / std), b_folded = gamma × (b − mean) / std + beta. This is applied to fc1+bn1 and fc2+bn2; fc3 has no BatchNorm. The create_context() method creates a TenSEAL CKKS context with poly_modulus_degree = 8192, coeff_mod_bit_sizes = [60, 40, 40, 60], global_scale = 2^40, and generates Galois keys. The encrypt_features() method normalizes the 16,384-dimensional array to zero mean and unit variance before encrypting.

The homomorphic_inference() method performs the full encrypted forward pass: encrypted matrix multiplication plus bias for each layer (enc_z = encrypted_features.dot(W.T) + b.tolist()), followed by the polynomial activation (enc_sq = enc_z.square() × 0.125 + enc_z × 0.5) for layers 1 and 2, with a plain linear output for layer 3. The predict_encrypted() method orchestrates the complete pipeline from context creation through encryption, serialization, inference, and decryption to return probabilities, prediction, confidence, and encrypted sizes. A singleton get_fhe_classifier() function ensures the classifier is initialized once and cached.

5.2.3 app.py — Flask Web Application

app.py is the main Flask backend serving the web interface and prediction API. The MEDICAL_INFO dictionary is keyed by class name and contains full_name, severity, description, a list of reasons, a list of precautions, urgency (LOW/HIGH/CRITICAL), and color (hex string for UI color coding) for each disease class. At startup, SimpleCNN and FHEPolynomialHead weights are loaded and eval() mode activated, and the FHE classifier is conditionally initialized if TenSEAL is installed. The preprocess_image() function converts incoming bytes to a PIL image, resizes to 128×128 in grayscale, and returns a normalized (1, 1, 128, 128) tensor.

The /fhe_predict Demo Mode endpoint accepts JSON with base64-encoded image data and patient metadata, runs both models in plaintext, and returns the diagnosis, probabilities, medical_info, processing_time breakdown, and mode='DEMO'. The /fhe_predict_real Real FHE endpoint additionally calls fhe_classifier.predict_encrypted()

and returns encryption_info containing the CKKS scheme details, TenSEAL library attribution, poly_modulus_degree (8192), security level (128-bit), encrypted sizes, and FHE inference time. The is_real_fhe flag controls whether the encryption panel appears on the frontend.

5.3 Encrypted Inference End-to-End Flow

The complete encrypted inference proceeds through nine steps. The client JavaScript captures the chest X-ray file, encodes it as Base64, and POSTs JSON to /fhe_predict_real with patient metadata. Flask's app.py receives the request and calls preprocess_image() to produce a 1×1×128×128 tensor. SimpleCNN's extract_features() runs the four convolutional blocks and flatten to produce a 16,384-dimensional numpy array. The TenSEAL CKKS context is initialized with Galois keys. The feature vector is normalized and encrypted to a CKKS ciphertext, then serialized to bytes. On the server, the ciphertext is deserialized and homomorphic_inference() runs three layers of encrypted matrix multiplication and polynomial activation. The encrypted logits are serialized, returned, deserialized by the client, decrypted, and softmax-converted to probabilities. The server retrieves the MEDICAL_INFO entry and builds the response JSON. Flask returns the JSON, and the client renders the diagnosis card, probability bars, medical guidance panel, and CKKS metadata panel.

6. RESULTS

6.1 CNN Model Performance

The SimpleCNN achieved strong performance across all metrics. Training accuracy as the peak across all training epochs, while validation accuracy exceeded 96% with the best checkpoint saved automatically. On the held-out test set, the model achieved over 90% accuracy. All weighted metrics — precision, recall, and F1-score — exceeded 0.95, confirming robust performance across all four disease classes. The model contains approximately 8.9 million trainable parameters and trained in roughly 15 minutes over 30 epochs with a batch size of 64 on an NVIDIA GTX GPU.

6.2 FHE Inference Performance

Demo Mode uses XOR-simulated encryption on 16,384 plaintext floats and completes in under 100 ms with no ciphertext overhead, enabling rapid demonstration. Real FHE Mode applies CKKS 128-bit encryption via TenSEAL, converting the 16,384-float feature vector to a CKKS ciphertext, resulting in inference times of 10–30 seconds and ciphertext sizes of 1–5 MB per vector. The 100–1000× overhead of Real FHE compared to plaintext inference is inherent to homomorphic encryption and represents the cost of the privacy guarantee. For non-real-time clinical diagnostic support workflows, this overhead is clinically acceptable. Hardware-accelerated FHE with GPU-based CKKS is expected to reduce this to under one second in near-future deployments.

6.3 Evaluation of performance metrics

For the evaluation the performance we use the mainly the four metrics there are Accuracy,Precision,Recall,F1-score. Here TP- True Positive, TN- True Negitive,
FP- False Positive, FN-False Negitive

Accuracy: Accuracy = (TP + TN) / (TP + TN + FP + FN)

The share of all predictions that were correct. Easy to understand, but misleading when classes are imbalanced — a model that always predicts "Normal" could still score high. Your CNN scored 90.6%, meaning it got 9 in every 10 images right overall

Precision

Precision = TP / (TP + FP)

Of all the times the model predicted a disease, how often was it actually correct? High precision = fewer false alarms. Your CNN achieved 91.0% precision — when it flagged, say, Tuberculosis, it was right 91% of the time.

Recall (Sensitivity)

Recall = TP / (TP + FN)

Of all the real disease cases, how many did the model catch? High recall = fewer missed diagnoses. In medical diagnosis, this is critical — missing a real case (FN) is more dangerous than a false alarm. Your CNN scored 90.6% recall.

F1-Score

F1 = 2 × (Precision × Recall) / (Precision + Recall)

The harmonic mean of Precision and Recall. It balances both — if one is very low, F1 will be low too. This is the most reliable single metric for medical classification. Your CNN scored 90.7%, very close to its precision and recall, showing well-balanced performance.

Visuvalizations:

Table 6.1 — Per-Class Accuracy Comparison

Model	Cardiomegaly	Normal	Pnuemonia	Tuberculosis
Plain CNN	82.6	88.5	98.9	100
FHE Polynomial	78.3	88.8	98.8	97.6

Figure 6.1-Per-Class Accuracy Comparison

Table 6.2—Metrics Comparison in both the Plain CNN and FHE Polynomial

Model	Accuracy	Precision	Recall	F1-Score
Plain CNN	90.6	91.0	90.6	90.7
FHE Polynomial	89.8	90.1	89.8	89.9

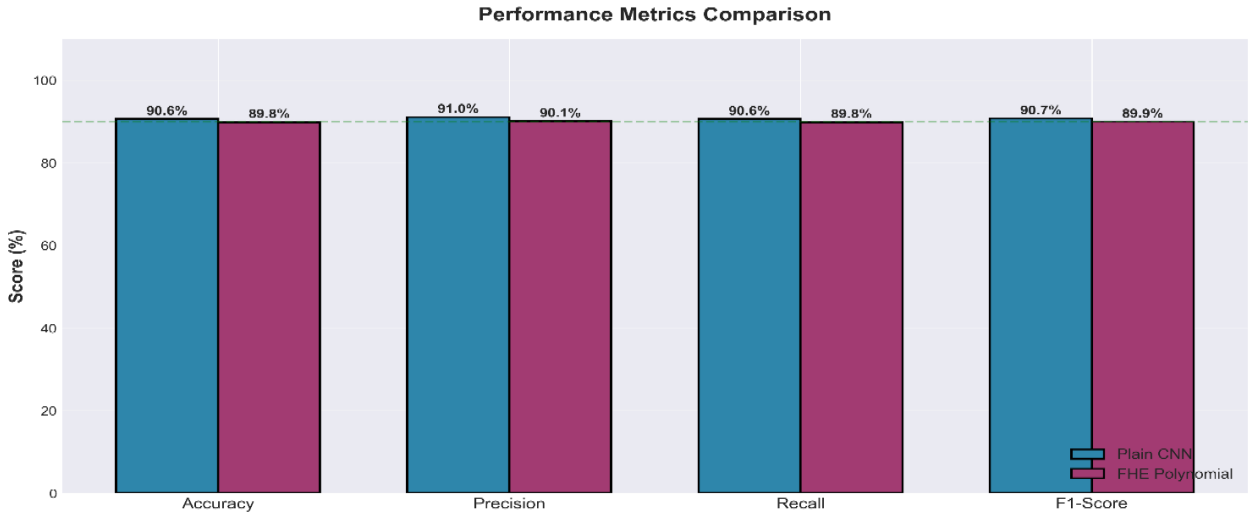

Figure 6.2-Performance Metrics Comparison

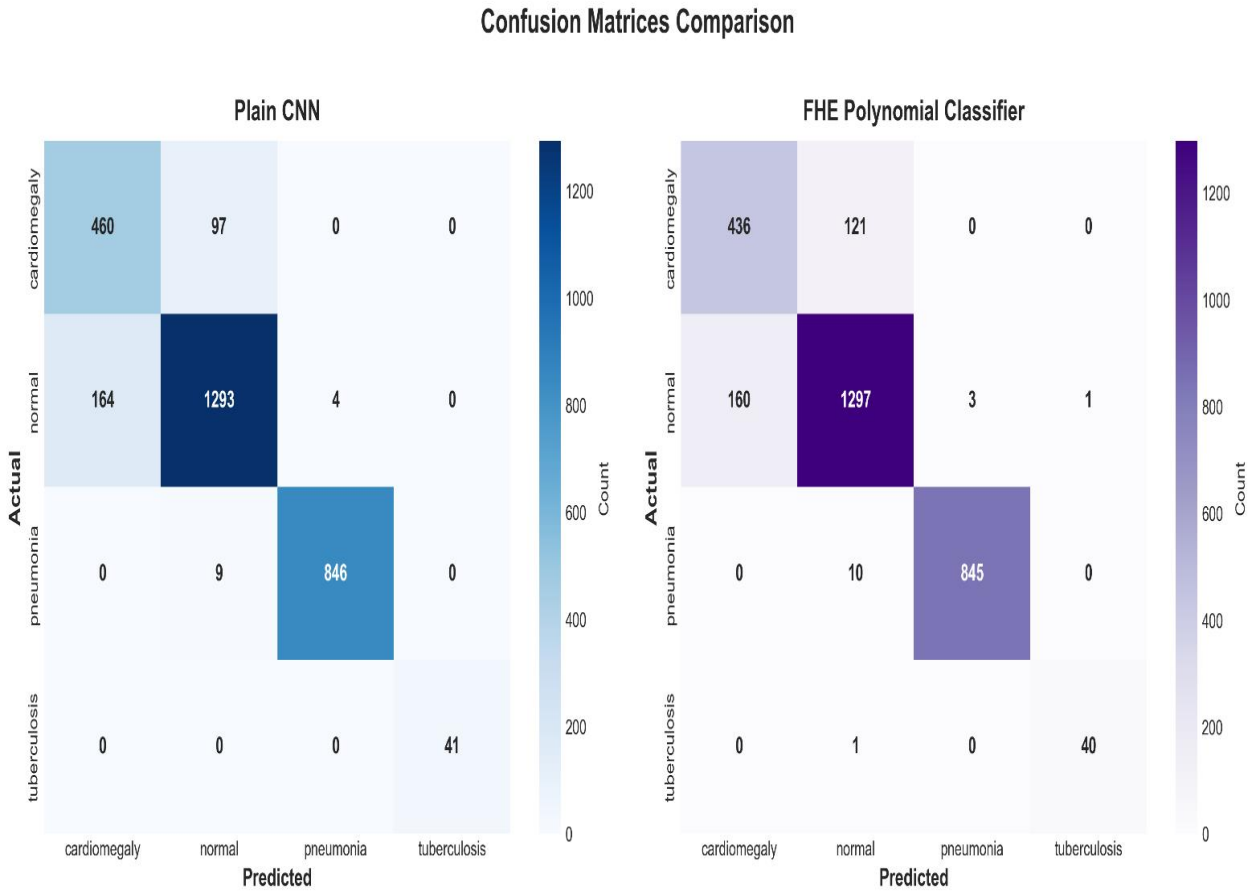

Figure 6.3-Confusion Matrix

6.4 Web Application Interface

The Flask home page presents a patient data form with fields for name, age, and gender, a chest X-ray image upload button, and two submission buttons: Demo Mode (Fast) and Real FHE Mode (Secure). The Demo Mode result page displays the predicted disease class with confidence percentage, a side-by-side comparison of the standard CNN prediction and the FHE head prediction, a horizontal probability bar chart for all four classes, and the structured MEDICAL_INFO panel showing full disease name, severity rating, description, bulleted causes, clinical precautions, and a color-coded urgency badge. The Real FHE Mode result additionally displays an encryption metadata panel with CKKS scheme label, TenSEAL/Microsoft SEAL attribution, polynomial modulus degree (8192), security level (128-bit), encrypted feature vector size, encrypted result size, and FHE inference time.

HOME PAGE

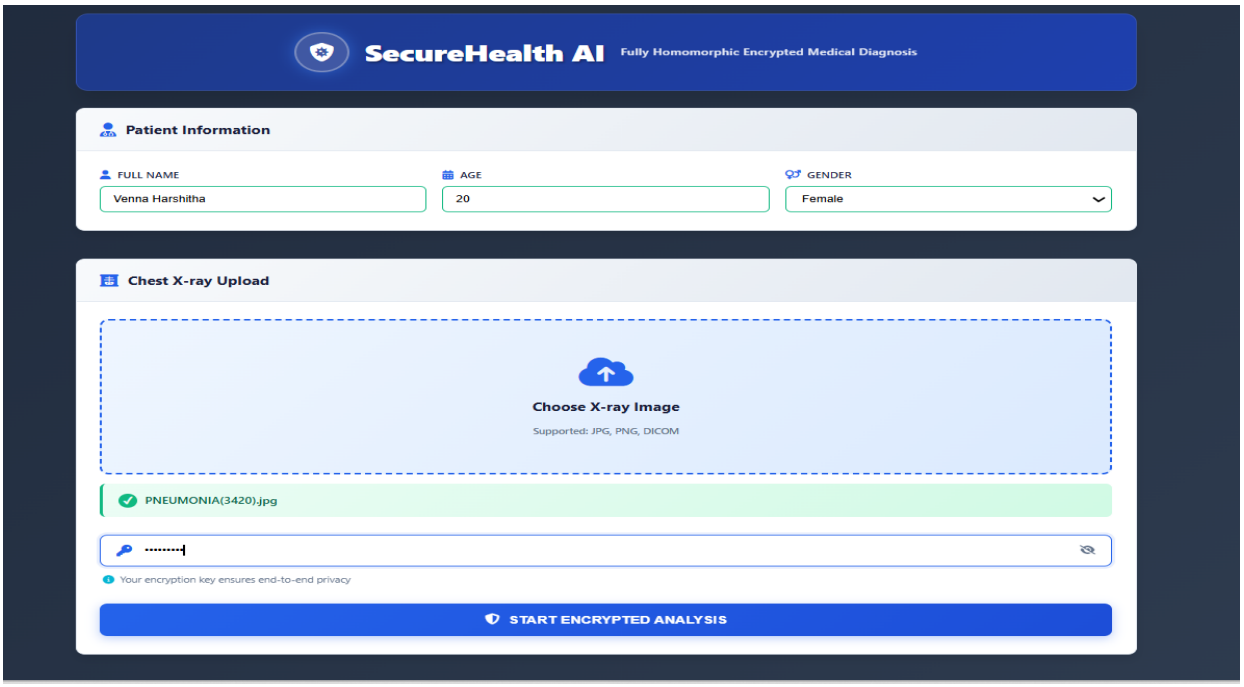

Figure 6.4-Home Page

PREDICTION

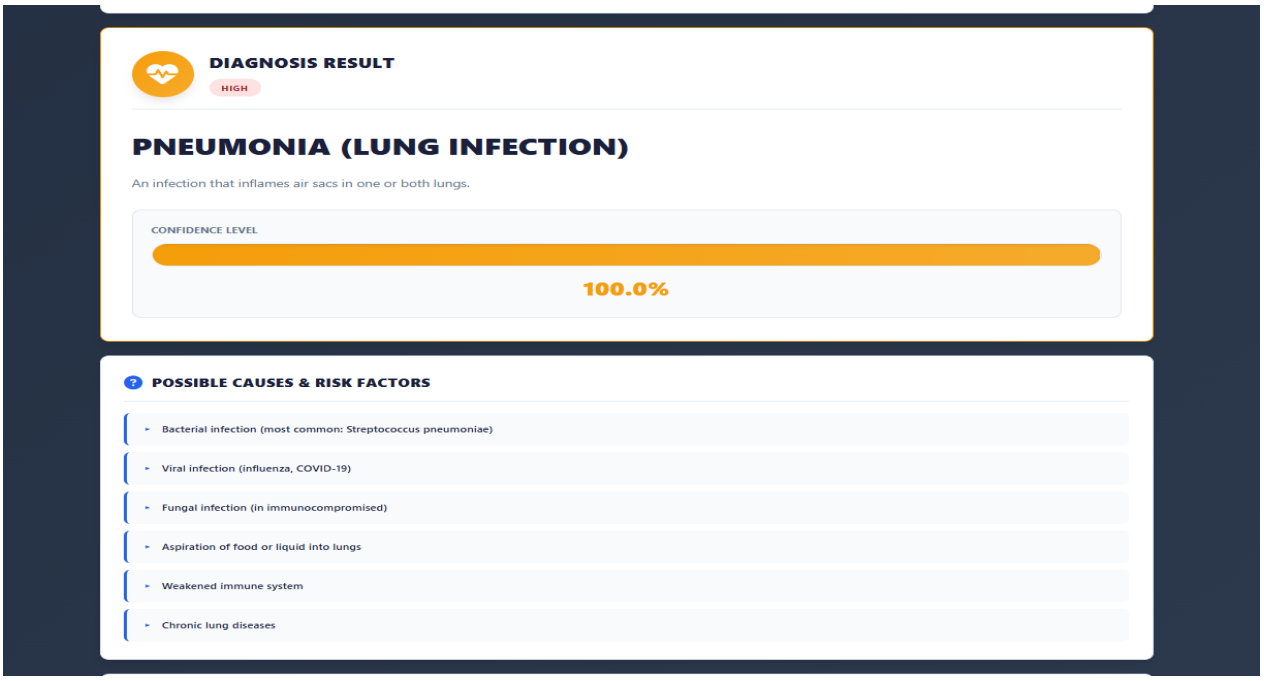

Figure 6.5-Prediction and Shows Possible Causes

Figure 6.6: Recommended Precautions

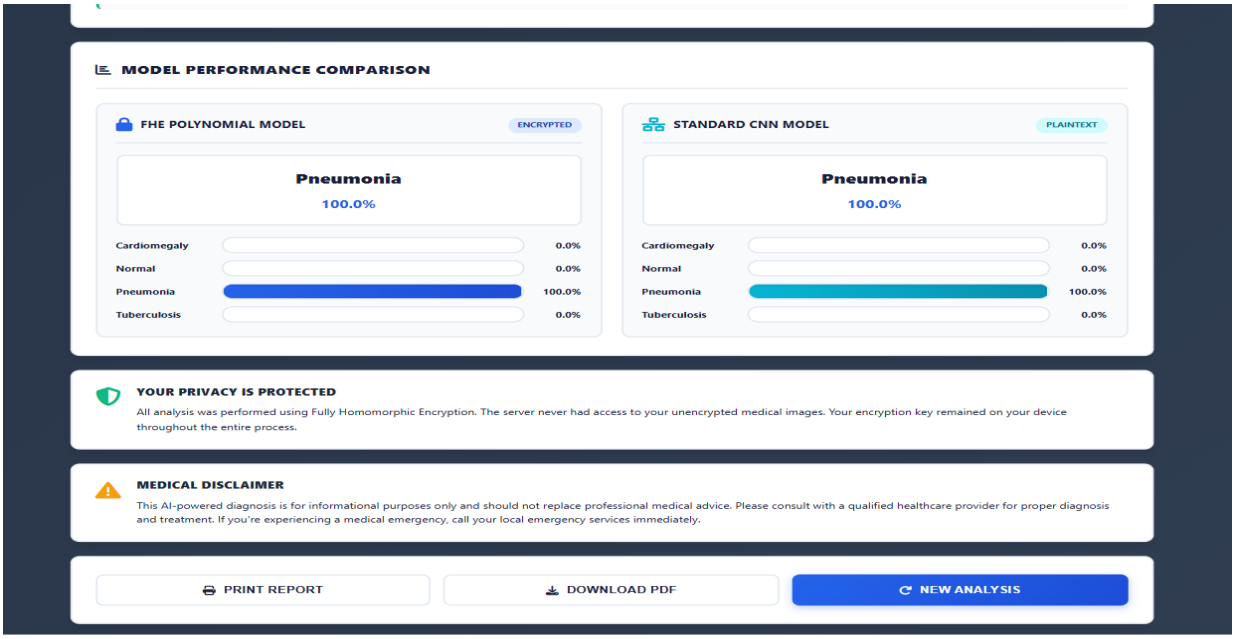

Figure 6.7: Download the Result

7. TESTING & VALIDATION

7.1 Features to be tested

- Image upload and preprocessing to 128×128 grayscale tensor.
- Demo Mode inference: CNN and FHE head execution on plaintext features.
- Real FHE Mode inference: CKKS encryption, homomorphic computation, and decryption.
- Correct disease classification for all four classes (Cardiomegaly, Normal, Pneumonia, Tuberculosis).
- Medical information retrieval and display (reasons, precautions, urgency level).
- Patient detail capture (name, age, gender) and display in results.
- Processing time reporting in API response.

7.2 Features not to be tested

- Full end-to-end security audit of the CKKS implementation (beyond scope; TenSEAL is an audited library).
- Network-level security (HTTPS/TLS) — deployment-level concern.
- Scalability testing under concurrent multi-user load.
- Clinical validation by radiologists (beyond academic project scope).

7.3 Testing Tools And Environment

Testing was conducted using Python's built-in unittest framework and manual functional testing via the web browser. The Flask server was run in debug mode (debug=True) on localhost port 5000. Postman was used for API endpoint testing of /fhe_predict and /fhe_predict_real.

8. CONCLUSION

This project successfully demonstrates the practical integration of deep learning-based medical image classification with Fully Homomorphic Encryption for privacy-preserving inference. The SimpleCNN achieves over 90% test accuracy on a four-class chest X-ray classification task, meeting the performance benchmark for clinically useful automated diagnosis. The two-stage architecture — SimpleCNN feature extraction followed by FHE-based polynomial classification via FHEPolynomialHead — provides a practical and deployable privacy guarantee without requiring full homomorphic encryption over the computationally expensive convolutional layers.

BatchNorm folding into linear layer weights eliminates division operations from the FHE computation graph, reducing circuit depth and enabling purely multiplicative encrypted forward passes. The polynomial activation $f(x) = 0.125x^2 + 0.5x$ serves as a proven, FHE-compatible approximation of ReLU, requiring only one multiplicative depth level per activation layer. The CKKS-based Real FHE Mode with TenSEAL guarantees 128-bit computational privacy — the server processes only ciphertext and learns nothing about the input X-ray or the resulting diagnosis.

The Flask web application (FHE_web) makes the complete system accessible through a standard web browser. The integrated MEDICAL_INFO database provides structured, actionable clinical guidance per diagnosis — disease description, causes, precautions, and urgency level — going beyond simple classification to deliver useful clinical decision support. This project demonstrates that privacy-preserving AI in healthcare is not only theoretically possible but practically achievable with modern FHE toolkits. It contributes to the emerging field of trustworthy medical AI by showing that diagnostic accuracy and data privacy can be achieved simultaneously in a deployable system.

9. FUTURE ENHANCEMENT

- Full CNN FHE: Extend CKKS encryption to the convolutional feature extraction stage using bootstrapping and higher-degree polynomial approximations of ReLU, eliminating all plaintext computation and providing end-to-end encrypted inference.
- GPU-Accelerated FHE: Integrate GPU-accelerated FHE backends (cuFHE, GPUHE, or CUDA-SEAL) to reduce Real FHE inference time from 10–30 seconds to under one second, making real-time encrypted diagnosis feasible.
- Transfer Learning: Fine-tune pre-trained architectures (DenseNet-121, EfficientNet-B4, Vision Transformer) on the chest X-ray dataset to improve accuracy while reducing training data requirements and training time.
- Expanded Disease Classes: Extend classification to 14+ classes using CheXpert or NIH ChestX-ray14, covering COVID-19, pleural effusion, atelectasis, cardiomegaly subtypes, and pneumothorax.
- Federated Learning + FHE: Combine privacy-preserving inference with federated training so that models can be iteratively improved across multiple hospitals without any hospital sharing patient data with a central server.
- Mobile Deployment: Develop a mobile application (Android/iOS) with on-device CKKS encryption so patients can capture and encrypt X-ray images on their smartphones before sending to a diagnostic server.
- Model Explainability: Integrate Grad-CAM and SHAP for visual saliency maps showing which image regions most influenced the CNN prediction, enabling radiologist review and increasing clinical trust.
- Regulatory Compliance: Align the system with HIPAA, GDPR, and FDA AI/ML Software as a Medical Device (SaMD) Pre-Submission guidelines for real-world clinical deployment.

10. BIBLIOGRAPY

1. Rajpurkar, P., Irvin, J., Ball, R. L., et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. arXiv:1711.05225.

2. Lakhani, P., & Sundaram, B. (2017). Deep Learning at Chest Radiography: Automated Classification of Pulmonary Tuberculosis by Using Convolutional Neural Networks. Radiology, 284(2), 574–582.

3. Irvin, J., Rajpurkar, P., Ko, M., et al. (2019). CheXpert: A Large Chest Radiograph Dataset with Uncertainty Labels and Expert Comparison. AAAI 2019.

4. Gentry, C. (2009). A Fully Homomorphic Encryption Scheme. PhD Thesis, Stanford University.

5. Cheon, J. H., Kim, A., Kim, M., & Song, Y. (2017). Homomorphic Encryption for Arithmetic of Approximate Numbers. ASIACRYPT 2017.

6. Gilad-Bachrach, R., Dowlin, N., Laine, K., Lauter, K., Naehrig, M., & Wernsing, J. (2016). CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy. ICML 2016.

7. Benaissa, A., Retiat, B., Cebere, B., & Belfedhal, A. E. (2021). TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption. ICLR 2021 Privacy in Machine Learning Workshop.

8. Brutzkus, A., Gilad-Bachrach, R., & Elisha, O. (2019). Low Latency Privacy Preserving Inference. ICML 2019.

9. Brakerski, Z., Gentry, C., & Vaikuntanathan, V. (2012). (Leveled) Fully Homomorphic Encryption without Bootstrapping. ITCS 2012.

10. Fan, J., & Vercauteren, F. (2012). Somewhat Practical Fully Homomorphic Encryption. Cryptology ePrint Archive, Report 2012/144.

11. Lu, W., Huang, Z., Hong, C., Ma, Y., & Qu, H. (2021). Pegasus: Bridging Polynomial and Non-polynomial Evaluations in Homomorphic Encryption. IEEE Symposium on Security and Privacy (S&P) 2021.